

# Gaussian Mixture Models

Hrachya Asatryan

29.04.2026

# The Hard Assignment Problem

- In standard K-Means, the responsibility  $r_{ik}$  (the assignment of point  $x_i$  to cluster  $k$ ) is strictly boolean:  $r_{ik} \in \{0, 1\}$ .
- **The Geometric Flaw:** If a point sits exactly equidistant between two centroids, K-Means forces a 1 or 0. It completely discards the model's uncertainty.
- By making hard assignments, K-Means assumes absolute confidence, even at the decision boundaries.
- **The Goal:** We want  $r_{ik}$  to be a continuous value between 0 and 1, representing the *degree of membership*, with the strict constraint that  $\sum_k r_{ik} = 1$  for any given point  $i$ .

# Soft K-Means: The Heuristic Approach

- How do we calculate a continuous  $r_{ik}$ ? We pass the negative squared distances through a softmax function.
- We introduce a **stiffness parameter** ( $\beta$ ):

## Soft Responsibility

$$r_{ik} = \frac{\exp(-\beta \|x_i - \mu_k\|^2)}{\sum_j \exp(-\beta \|x_i - \mu_j\|^2)}$$

- **The Inverse Temperature Analogy:**
  - As  $\beta \rightarrow \infty$ , the function becomes a rigid step function (Standard Hard K-Means).
  - As  $\beta \rightarrow 0$ , the function flattens. Every point is equally assigned to every cluster (Maximum Entropy).

# Soft K-Means: The Centroid Update

- Because our responsibilities are now continuous fractions, the centroid update step must change.
- The centroid  $\mu_k$  is no longer the simple mean of a discrete, isolated set of points.
- It is now the **weighted average** of *all* points in the entire dataset, weighted by their responsibility to cluster  $k$ :

## Weighted Centroid

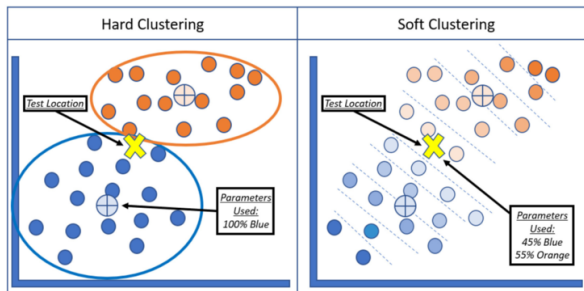
$$\mu_k = \frac{\sum_{i=1}^N r_{ik} x_i}{\sum_{i=1}^N r_{ik}}$$

# Soft K-Means: The Centroid Update

- Because our responsibilities are continuous fractions, the centroid  $\mu_k$  is the **weighted average** of *all* points in the dataset:

## Weighted Centroid

$$\mu_k = \frac{\sum_{i=1}^N r_{ik} x_i}{\sum_{i=1}^N r_{ik}}$$



# Why stop at heuristics?

- Soft K-Means gives us fractional assignments, but these are *not* true probabilities. They are just distances passed through an arbitrary exponential smoothing function.
- Furthermore, Soft K-Means still relies on Euclidean distance  $\|x_i - \mu_k\|^2$ .
- **The Geometric Consequence:** It still fundamentally assumes that all clusters are perfectly spherical and share the exact same radius.
- **The Question:** If we want true probabilities, and we want flexible cluster shapes (ellipses of varying densities), why not just build a formal Generative Model?

# Generative View of Clustering

- We need a sensible, mathematical measure of what it means to cluster the data well.
  - This makes it possible to judge different methods objectively.
  - It may help us decide on the number of clusters (Model Selection).
- **The Paradigm Shift:** Imagine that the data was produced by a hidden *generative model*.
- If we can define how the universe generated the data, we can adjust our model parameters using **Maximum Likelihood**—maximizing the probability that our model would produce exactly the data we observed.

# The Generative Model: GMMs

- We assume the data  $D$  is generated by a mixture of  $K$  different Multivariate Normal distributions. Let a data point  $x$  be generated as follows:
  - 1 Choose a cluster  $z$  from  $\{1, \dots, K\}$  such that  $p(z = k) = \pi_k$ .
  - 2 Given  $z$ , sample  $x$  from a Gaussian distribution  $\mathcal{N}(x|\mu_k, \Sigma_k)$ .
- This defines the joint distribution  $p(z, x) = p(z)p(x|z)$  with parameters  $\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$ .
- The **true marginal probability** of observing any point  $x$  is the sum over all possible hidden clusters:

## Gaussian Mixture Model

$$p(x) = \sum_z p(z, x) = \sum_{k=1}^K \pi_k \mathcal{N}(x|\mu_k, \Sigma_k)$$

# The Mathematical Wall

- $p(z = k|x)$  can be computed using Bayes rule and tells us the *true posterior probability* that  $x$  came from the  $k$ -th cluster (our ideal responsibilities!).
- To find the best parameters, we want to maximize the Log-Likelihood of our entire dataset  $X$ :

## The Intractable Log-Likelihood

$$\ln p(X|\theta) = \sum_{i=1}^N \ln \left( \sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k) \right)$$

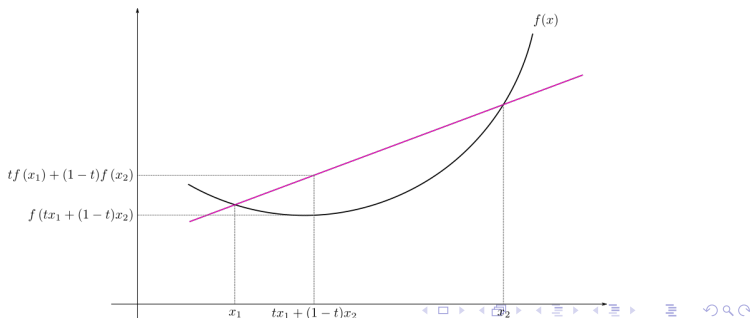
- **The Wall:** Notice the sum over  $k$  is *inside* the logarithm.
- We cannot push the logarithm inside the sum. Taking the derivative of the log of a sum yields equations that have no closed-form algebraic solution. We cannot just calculate the answer.

# The Mathematical Trick: Jensen's Inequality

- To bypass the sum trapped inside the logarithm, we construct a **lower bound** to our objective function.
- **Jensen's Inequality:** For any strictly concave function  $f$  (like a logarithm) and any random variable  $Y$ :

$$f(\mathbb{E}[Y]) \geq \mathbb{E}[f(Y)]$$

- *"The log of an expectation is greater than or equal to the expectation of the log."*



# A Quick Detour: What is KL Divergence?

- Before we can optimize our Gaussian Mixture Model, we need a mathematical way to measure the "distance" between two probability distributions.
- **Kullback-Leibler (KL) Divergence** (also called Relative Entropy) is exactly that tool.
- If we have a true probability distribution  $P(x)$ , and we try to approximate it with a model distribution  $Q(x)$ , the KL Divergence measures the *information lost* when we use  $Q$  to approximate  $P$ .

## The Property of Non-Negativity

$$D_{KL}(P||Q) \geq 0$$

It is exactly zero if and only if  $P$  and  $Q$  are identical distributions.

# The Formula and the "Two Pipes"

- **Why  $||$  instead of a comma?** The two pipes explicitly denote that this is **not a symmetric distance metric**.
- In standard geometry, the distance from A to B is the same as B to A. In probability theory, the divergence of  $P$  from  $Q$  is *not* the same as  $Q$  from  $P$ .

$$D_{KL}(P||Q) \neq D_{KL}(Q||P)$$

- **The Discrete Formula:**

$$D_{KL}(P||Q) = \sum_x P(x) \log \left( \frac{P(x)}{Q(x)} \right)$$

- **The Continuous Formula:**

$$D_{KL}(P||Q) = \int P(x) \log \left( \frac{P(x)}{Q(x)} \right) dx$$

# The Intuition: Why does the math work?

- Look at the logarithm in the formula:  $\log\left(\frac{P(x)}{Q(x)}\right)$ .
- **Case 1:  $P(x)$  and  $Q(x)$  are similar.** The ratio is  $\approx 1$ . The logarithm of 1 is 0. No divergence is added.
- **Case 2:  $P(x)$  is high, but  $Q(x)$  is low.** The ratio is large. The logarithm is a large positive number. We multiply by  $P(x)$  (which is high), resulting in a massive penalty!
- **Case 3:  $P(x)$  is low, but  $Q(x)$  is high.** The ratio is close to 0. The logarithm is a large negative number. *But*, we multiply it by  $P(x)$  (which is near 0), so the penalty is suppressed.
- **Conclusion:**  $D_{KL}(P||Q)$  heavily penalizes models ( $Q$ ) that assign low probability to events that actually happen frequently in reality ( $P$ ).

# Why do we care right now?

- We are trying to find the hidden clusters in our data (the Latent Variables  $Z$ ).
- We have the **true posterior probability**  $P(Z|X, \theta)$  (which is trapped behind intractable math).
- We are going to introduce an **arbitrary guess distribution**  $q(Z)$ .
- **The E-Step of the EM Algorithm:** We are literally just minimizing  $D_{KL}(q(Z)||P(Z|X, \theta))$ . We tweak our guess  $q(Z)$  until the divergence drops to 0, meaning our guess perfectly matches the true hidden cluster assignments!

# Constructing the Evidence Lower Bound (ELBO)

- Let's introduce an arbitrary probability distribution  $q(Z)$  over our hidden latent variables (the cluster assignments).
- We multiply and divide the inside of our log-likelihood by  $q(Z)$  to create an Expectation:

$$\ln p(X) = \ln \sum_Z p(X, Z) = \ln \sum_Z q(Z) \frac{p(X, Z)}{q(Z)} = \ln \mathbb{E}_q \left[ \frac{p(X, Z)}{q(Z)} \right]$$

- Now, we apply **Jensen's Inequality** to push the logarithm *inside* the expectation:

## The ELBO ( $\mathcal{L}$ )

$$\ln p(X) \geq \sum_Z q(Z) \ln \frac{p(X, Z)}{q(Z)} = \mathcal{L}(q, \theta)$$

- We now have a sum of logs instead of a log of sums!

# The Anatomy of ELBO and KL Divergence

- What is the exact mathematical gap between our true log-likelihood  $\ln p(X)$  and our new lower bound  $\mathcal{L}(q, \theta)$ ?
- Through algebraic expansion, the exact relationship is:

## The Fundamental Equation of Variational Inference

$$\ln p(X) = \mathcal{L}(q, \theta) + D_{KL}(q(Z) || p(Z|X, \theta))$$

- **Kullback-Leibler (KL) Divergence:** A measure of how different  $q(Z)$  is from the true posterior  $p(Z|X)$ .
- $D_{KL}$  is strictly  $\geq 0$ , and equals exactly 0 if and only if  $q(Z) = p(Z|X)$ .

# The E-Step (Expectation)

- The EM algorithm is a form of **Coordinate Ascent**. We alternate optimizing  $q$  and  $\theta$ .
- **Goal of the E-Step:** Freeze the model parameters  $\theta$ . We want to make our lower bound  $\mathcal{L}$  as tight as possible against the true likelihood.
- Because  $\ln p(X)$  is fixed (since  $\theta$  is frozen), maximizing  $\mathcal{L}$  means we must **minimize** the KL divergence.

## The E-Step Solution

We set  $q(Z) = p(Z|X, \theta)$  to force  $D_{KL} = 0$ .

- *Intuition:* We update our arbitrary distribution  $q(Z)$  to exactly match the true posterior probabilities (the "responsibilities") of the cluster assignments given our current parameters.

# The M-Step (Maximization)

- **Goal of the M-Step:** Now, we freeze our cluster assignments  $q(Z)$  (the responsibilities). We want to update our parameters  $\theta$  to push the true likelihood up.
- Since  $D_{KL} \geq 0$ , maximizing the lower bound  $\mathcal{L}$  guarantees we are also pushing up the true log-likelihood  $\ln p(X)$ .
- We maximize  $\mathcal{L}(q, \theta)$  with respect to  $\theta$ :

## The M-Step Objective

$$\theta^{(new)} = \arg \max_{\theta} \sum_Z q(Z) \ln p(X, Z | \theta)$$

- Because the logarithm is now pushing directly against the joint probability, taking the derivatives with respect to  $\mu$ ,  $\Sigma$ , and  $\pi$  yields beautiful, closed-form solutions!

# Applying EM to GMMs: The M-Step Updates

- When we substitute the Gaussian probability density function into our M-Step objective and take the derivatives, we obtain exact, closed-form updates for our parameters.
- Let  $N_k = \sum_{i=1}^N r_{ik}$  be the "effective number of points" assigned to cluster  $k$ .

- **1. The Prior (Mixing Coefficient):**

$$\hat{\pi}_k = \frac{N_k}{N}$$

- **2. The Mean:**

$$\hat{\mu}_k = \frac{1}{N_k} \sum_{i=1}^N r_{ik} x_i$$

- **3. The Covariance:**

$$\hat{\Sigma}_k = \frac{1}{N_k} \sum_{i=1}^N r_{ik} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T$$

# Let's DO IT: A 1D EM Walkthrough

- Let's trace the math on a simple 1D dataset of 10 points:

$$X = \{1, 2, 2.5, 3, 8, 8.5, 9, 9.5, 10, 10.5\}$$

- Visually, we have a small cluster on the left and a larger cluster on the right. Let's initialize  $K = 2$  Gaussians:

## Initialization (Iteration 0)

- Gaussian 1 ( $G_1$ ):  $\mu_1 = 3.0, \sigma_1^2 = 1.0, \pi_1 = 0.5$
  - Gaussian 2 ( $G_2$ ):  $\mu_2 = 6.0, \sigma_2^2 = 1.0, \pi_2 = 0.5$
- Notice that  $G_2$  is initialized at  $x = 6$ , completely in the empty space between the two true clusters!

# Iteration 1: The E-Step

- We freeze the parameters and calculate the responsibilities  $r_{i,k}$  for every point.
- Let's look at the data point  $x_5 = 8$ :

$$\text{Claim of } G_1 = \pi_1 \cdot \mathcal{N}(8|3, 1) \approx 0.5 \cdot (1.48 \times 10^{-6}) \approx 0$$

$$\text{Claim of } G_2 = \pi_2 \cdot \mathcal{N}(8|6, 1) \approx 0.5 \cdot 0.054 = 0.027$$

- We normalize these claims to get the true responsibility:

$$r_{5,2} = \frac{0.027}{0 + 0.027} = 1.0$$

- **Result:** Even though  $G_2$  is sitting in empty space, it is technically closer to the right-side points. It takes  $\approx 100\%$  ownership of  $\{8, 8.5, 9, 9.5, 10, 10.5\}$ .

# Iteration 1: The M-Step

- Now we freeze the responsibilities and update the parameters.
- Let's update the mean of  $G_2$  ( $\mu_2$ ).
- Because  $G_1$  claimed the left points, their  $r_{i,2}$  values are 0. We only sum the points  $G_2$  claimed:

## Updating $\mu_2$

$$\hat{\mu}_2 = \frac{\sum r_{i,2} x_i}{\sum r_{i,2}} \approx \frac{8 + 8.5 + 9 + 9.5 + 10 + 10.5}{6} = 9.25$$

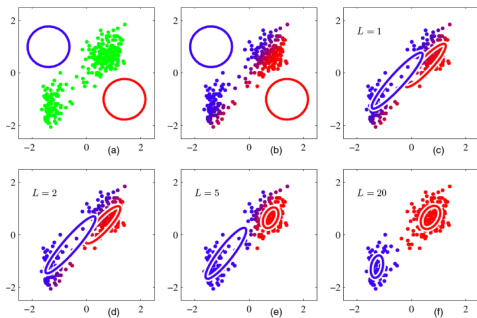
- **The Physics of M-Step:**  $\mu_2$  is aggressively violently pulled from the empty space at  $x = 6$  directly into the center of mass of the right cluster at  $x = 9.25$ !
- The variance  $\sigma_2^2$  will also expand to  $\approx 0.72$  to perfectly cover the spread of those 6 points.

# Convergence

- By Iteration 2, the E-Step will calculate responsibilities based on the new, highly accurate means ( $\mu_1 \approx 2.12, \mu_2 \approx 9.25$ ).
- Because the Gaussians are now sitting directly on top of the data, the responsibilities solidify.
- The changes in  $\mu$  and  $\sigma$  in Iteration 3 will be microscopic ( $< 10^{-4}$ ).
- **Algorithm Terminated.** The model has successfully identified the hidden latent variables (the clusters) and the generative parameters, completely unsupervised, purely by minimizing the KL Divergence iteratively.

# Convergence

- By Iteration 2, the E-Step calculates responsibilities based on the new, highly accurate means.
- Because the Gaussians are now sitting directly on top of the data, the responsibilities solidify.
- **Algorithm Terminated.** The model has successfully identified the hidden latent variables completely unsupervised.



# The "Soft K-Means" Revelation

- Look closely at the update equation for the Gaussian mean:

## GMM Mean Update

$$\hat{\mu}_k = \frac{1}{N_k} \sum_{i=1}^N r_{ik} x_i$$

- This is mathematically identical to the Soft K-Means centroid update we defined at the very beginning of the lecture!
- Instead of a hard boolean assignment (1 or 0), the center of the Gaussian is pulled by the fractional probability (the responsibility  $r_{ik}$ ) of every point in the dataset.
-

# The Grand Unification: K-Means as a GMM

- We can formally prove that standard, Hard K-Means is just a highly restricted special case of a Gaussian Mixture Model.
- Under what exact conditions does a GMM collapse into K-Means?
  - 1 **Spherical Restriction:** We force all covariance matrices to be exactly equal and perfectly spherical:  $\Sigma_k = \epsilon I$ .
  - 2 **The Variance Limit:** We take the limit as the variance approaches zero ( $\epsilon \rightarrow 0$ ).
- **The Mathematical Result:** As the Gaussians become infinitely narrow, the responsibility  $r_{ik}$  for the closest centroid approaches exactly 1, and all others approach exactly 0.
- The soft probabilities snap into hard assignments, and the EM algorithm becomes the standard K-Means algorithm!

# Practical Nightmare 1: Singularities

- What happens if, during the EM updates, a Gaussian centroid lands exactly on top of a single, isolated data point?
- The "effective number of points"  $N_k$  for that cluster becomes 1.
- Look at the variance update:  $\hat{\Sigma}_k = \frac{1}{1}(x_i - \mu_k)^2$ . Since  $\mu_k = x_i$ , the variance shrinks exactly to 0.
- **The Infinite Likelihood Problem:** Look at the Gaussian density function:  $\frac{1}{\sqrt{2\pi\sigma^2}} \exp(\dots)$ . As  $\sigma \rightarrow 0$ , the probability density explodes to  $+\infty$ .
- The EM algorithm will violently crash, assigning infinite likelihood to a single point and failing to model the rest of the distribution.

# Fixing Singularities: Regularization

- To prevent the covariance matrices from collapsing to zero, we must intervene mathematically.
- **The Fix:** We add a very small, positive constant ( $\epsilon$ ) to the diagonal of the covariance matrix during every M-step:

## Regularized Covariance

$$\hat{\Sigma}_k = \hat{\Sigma}_k + \epsilon I$$

- *Callback:* This is the exact same mathematical trick used in Ridge Regression to prevent exploding weights, and in Regularized Discriminant Analysis!

## Practical Nightmare 2: Choosing 'K'

- In K-Means, we used the Elbow Method or Silhouette Score to guess the optimal number of clusters.
- Because a GMM is a generative model, it outputs a true numerical Log-Likelihood for the entire dataset.
- **The Trap:** Why can't we just try  $K = 2, 3, 4, \dots$  and pick the  $K$  that gives the highest Log-Likelihood?
- Because adding more Gaussians will *always* increase the likelihood! A GMM with  $K = N$  (one Gaussian per data point) has infinite likelihood. Using raw likelihood inherently leads to massive overfitting.

# Gaussian Mixture Models: Pros & Cons

## Pros:

- Can handle non-spherical clusters (elliptical, rotated) via the covariance matrix.
- Provides soft (probabilistic) assignments of clusters.
- Naturally handles cluster overlapping.
- Less sensitive to the scale of the data compared to distance-based K-Means.

## Cons:

- The EM algorithm is computationally heavy!
- Highly sensitive to initialization (can get stuck in local optima).
- Sensitive to noise and extreme outliers.
- Lack of model interpretability in high dimensions.
- Convergence to the global optimum is not guaranteed!

# Information Criteria (AIC & BIC)

- To find the true optimal  $K$ , we must penalize the model for being too complex.
- Let  $\hat{L}$  be the maximum likelihood, and  $p$  be the total number of parameters in the model.

## Akaike Information Criterion (AIC)

$$AIC = 2p - 2\ln(\hat{L})$$

## Bayesian Information Criterion (BIC)

$$BIC = p\ln(N) - 2\ln(\hat{L})$$

- **Usage:** We plot  $K$  against these scores and look for the **absolute minimum**. BIC penalizes complex models much more harshly than AIC due to the  $\ln(N)$  term.



# The Parametric Limitation

- A Gaussian Mixture Model is a *parametric* model. It forces us to declare  $K$  upfront.
- If we guess  $K = 3$  but the data is shaped like a continuous, swirling spiral, the GMM fails—it will just clumsily tile the spiral with three elliptical blobs.
- **The Question:** Can we estimate the probability density function  $p(x)$  without making *any* assumptions about the global shape of the data or the number of clusters?
- **The Naive Solution:** The Histogram.
- *Flaws:* Histograms are discontinuous (blocky), and their shape changes wildly depending on exactly where you arbitrarily place the bin edges.

# The Parzen-Rosenblatt Window (KDE)

- **The Intuition:** Instead of dropping points into fixed, rigid bins, what if we drop a continuous, smooth "mini-probability distribution" (a **kernel**) exactly on top of *every single data point*?
- By summing up all these individual kernels, we create a smooth, continuous density landscape.

## Kernel Density Estimation (KDE)

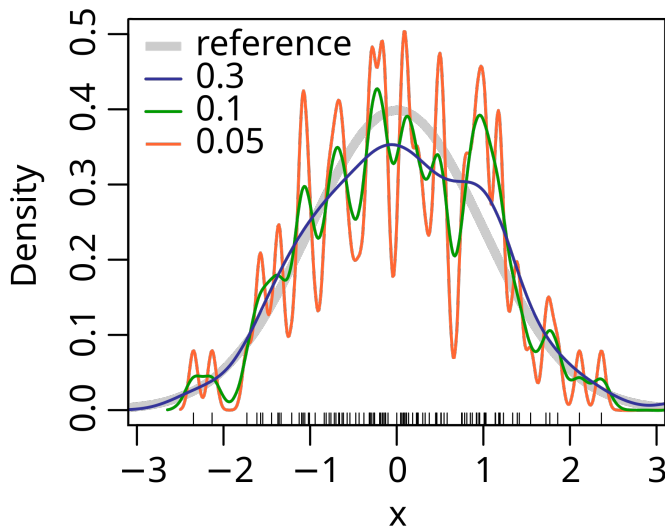
$$\hat{f}_h(x) = \frac{1}{N} \sum_{i=1}^N K_h(x - x_i) = \frac{1}{Nh} \sum_{i=1}^N K\left(\frac{x - x_i}{h}\right)$$

- **Components:**

- $N$ : Total number of training points.
- $K$ : The Kernel function (usually a standard Gaussian).
- $h$ : The **Bandwidth** parameter (controls the width of the kernel).



# The Parzen-Rosenblatt Window (KDE)



# The Grand Unification: KDE is a GMM!

- Look at the formula for a KDE using a Gaussian kernel, and compare it to the formula for a GMM.
- **The Revelation:** A Kernel Density Estimator is mathematically identical to a Gaussian Mixture Model where:
  - 1 The number of clusters is equal to the number of data points ( $K = N$ ).
  - 2 Every prior probability  $\pi_i$  is exactly  $\frac{1}{N}$ .
  - 3 The mean of each Gaussian  $\mu_i$  is permanently fixed to the exact coordinates of the training point  $x_i$ .
  - 4 The covariance matrix of every Gaussian is locked to  $\Sigma_i = h^2 I$ .
- We completely bypass the EM algorithm because there is nothing to learn! The data *is* the model.

# The Bias-Variance Tradeoff in KDE

- Because KDE has no weights or means to optimize, the entire model hinges on the single hyperparameter  $h$  (the bandwidth).
- **High Bandwidth (Large  $h$ ):** The kernels are wide and flat. They overlap massively, smoothing out the entire space.
  - *High Bias, Low Variance.* (Underfitting: Everything merges into a single blob).
- **Low Bandwidth (Small  $h$ ):** The kernels are narrow spikes. They don't overlap.
  - *Low Bias, High Variance.* (Overfitting: The density is just a series of isolated delta spikes exactly at the training points).

•

# Summary: The Density Landscape

- You have now seen the complete evolution of generative density estimation:
  - ① **Single Gaussian (GDA):** Assuming the whole world is one shape. (Maximum bias, infinite efficiency).
  - ② **GMM (EM Algorithm):** Assuming the world is made of  $K$  shapes. We mathematically discover their centers and widths.
  - ③ **KDE:** Assuming the world is too complex to model parametrically. We make every single data point its own shape. (Maximum variance, perfectly captures reality).
- **Next Lecture:** What if we don't care about probabilities, but just want to isolate dense regions from noise? We move to **DBSCAN** and **Spectral Clustering**.